第七章 驱动架构

本章内容

- Linux 2.6 风格的总线/设备/驱动模型
- 中断上半部与 workqueue 下半部
- 控制台、Mini UART 与 TTY
- USB 与 HID 键盘：描述符、枚举、split 事务、URB、真机排错
- I2C（BSC）
- 网络协议栈
- SDIO 无线网卡 BCM43430/43455
- USB 无线网卡 MT7601U
- CSI-2 摄像头 OV5647：D-PHY、CSI-2 包、Unicam FIFO、DMA 双缓冲与帧通知，HTTP MJPEG 推流及其流水线提速

7.1 设备模型

7.1.1 为什么不再只用 devsw[]

原版 xv6 只有一张字符设备函数表：`inode.major` 直接索引 `devsw[major].read/write`。它够用来驱动一个控制台，但表达不了“硬件设备”“驱动程序”“总线”以及“二者何时绑定”。随着驱动从一个变成十几个，项目引入了 Linux 2.6 设备模型的最小子集（`kernel/device.h`）：

```
struct bus_type {
  char *name;
  int  (*match)(struct device *dev, struct device_driver *drv);
  int  (*probe)(struct device *dev);
  void (*remove)(struct device *dev);
};
struct device_driver {
  char *name;  struct bus_type *bus;
  int  (*probe)(struct device *dev);
  void (*remove)(struct device *dev);
};
struct device {
  char *name;  int id;  struct device *parent;  struct bus_type *bus;
  struct resource resource[DEVICE_RES_MAX];  int nresource;  // MMIO、IRQ
  struct device_driver *driver;  void *driver_data;
};
struct file_operations {
  int  (*read)(int user_dst, uint64 dst, int n);
  int  (*write)(int user_src, uint64 src, int n);
  int  (*open)(struct file *f);                 // 可选：每次 open 的私有状态
  void (*release)(struct file *f);
  int  (*fread)(struct file *f, int user_dst, uint64 dst, int n);
};
```

设备和驱动可以按任意顺序注册：先注册的一方等着，后注册的一方触发总线扫描，`match()` 成功就调用驱动的 `probe()`。驱动在 `probe()` 中初始化硬件，然后用 `register_chrdev(major, name, fops)` 注册字符设备，或者 `register_netdev()` 注册网卡。